

GPU Scheduling in Kubernetes

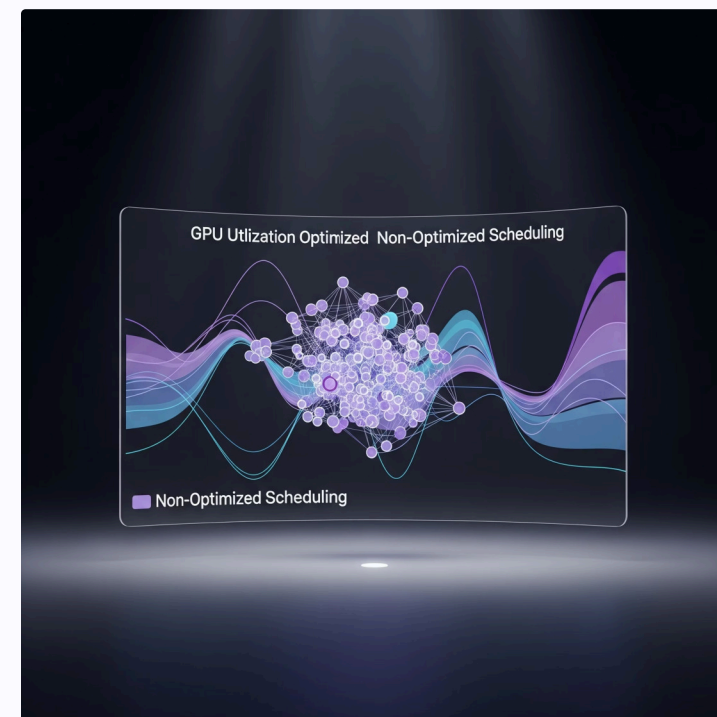
A practical guide for AI infrastructure teams and Kubernetes engineers



Why GPU Scheduling Matters

GPUs are expensive, scarce resources that require **efficient allocation** for AI workloads. The standard Kubernetes scheduler isn't GPU-aware, necessitating special configurations to:

- Properly assign GPUs to pods
- Prevent resource underutilization
- Avoid resource conflicts between workloads
- Maximize return on GPU infrastructure investment



Kubernetes and GPU Support



NVIDIA Device Plugin

Enables Kubernetes to recognize and manage NVIDIA GPUs as schedulable resources



Resource Requests

Pods request GPUs like CPUs/Memory through resource specifications



Cluster Support

Works across single-node and multi-node clusters for distributed AI workloads

This infrastructure is essential for both AI inference and training workloads, ensuring pods are scheduled on nodes with appropriate GPU resources.

Requesting a GPU in a Pod

```
resources:  
  limits:  
    nvidia.com/gpu: 1
```

This specification:

- Guarantees **exactly one GPU** for the pod
- Cannot request fractional GPUs (except with MIG)
- Scheduler ensures pod lands on a GPU-enabled node
- Prevents other pods from using the same GPU



Example – GPU Pod Specification

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  containers:
  - name: gpu-container
    image: pytorch/pytorch:latest
  resources:
    limits:
      nvidia.com/gpu: 1
```

This specification creates a pod that:

Runs a PyTorch container

Using the official PyTorch image with CUDA support

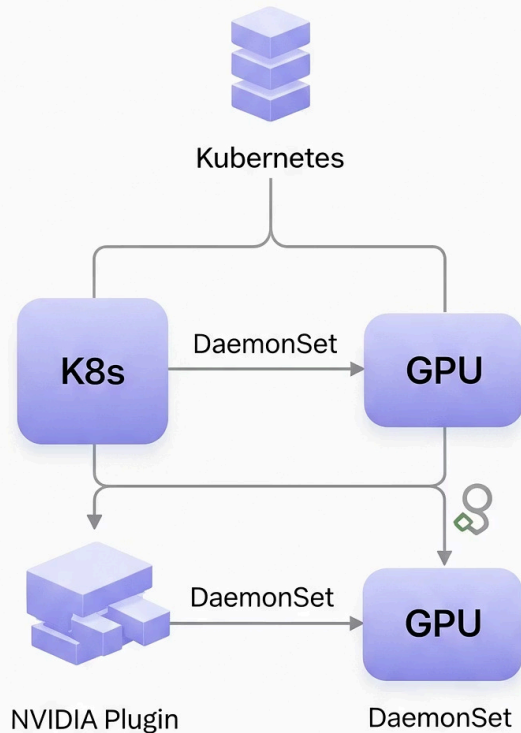
Requests a single GPU

Guarantees exclusive access to one NVIDIA GPU for AI workloads

Gets scheduled appropriately

K8s ensures this pod lands on a node with an available GPU

NVIDIA Device Plugin



NVIDIA Device Plugin

Key Components

- Deploys as a **DaemonSet** across all GPU nodes
- Advertises available GPUs to the Kubernetes scheduler
- Handles GPU discovery and initialization on each node

Hardware Support

- NVIDIA A100, H100, and other datacenter GPUs
- Multi-Instance GPU (MIG) technology
- Multi-GPU configurations (NVLink, etc.)

❏ Without this plugin, Kubernetes cannot schedule workloads on GPUs!

Multi-GPU & MIG Scheduling

Multi-GPU Requests

```
resources:  
limits:  
  nvidia.com/gpu: 4
```

Assigns 4 full GPUs to a single pod, ideal for distributed training workloads.

MIG Slices for Inference

```
resources:  
limits:  
  nvidia.com/mig-1g.5gb: 2
```

Requests 2 MIG slices (1 compute instance with 5GB memory each)



MIG (Multi-Instance GPU) enables [fractional GPU allocation](#), significantly improving resource utilization for inference workloads that don't require a full GPU.

Scheduling Across Nodes

1

Node Selectors

```
nodeSelector:  
  accelerator: nvidia-gpu
```

Ensures pods run only on labeled GPU nodes

2

Taints and Tolerations

```
tolerations:  
- key: "gpu"  
  operator: "Exists"  
  effect: "NoSchedule"
```

Reserves GPU nodes for GPU workloads only

3

Affinity Rules

```
affinity:  
  nodeAffinity:  
  
    requiredDuringSchedulingIgnoredDuring  
    Execution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: gpu-type  
            operator: In  
            values:  
              - "a100"
```

Fine-grained control for specific GPU types

These mechanisms ensure workloads run on appropriate hardware across your cluster.

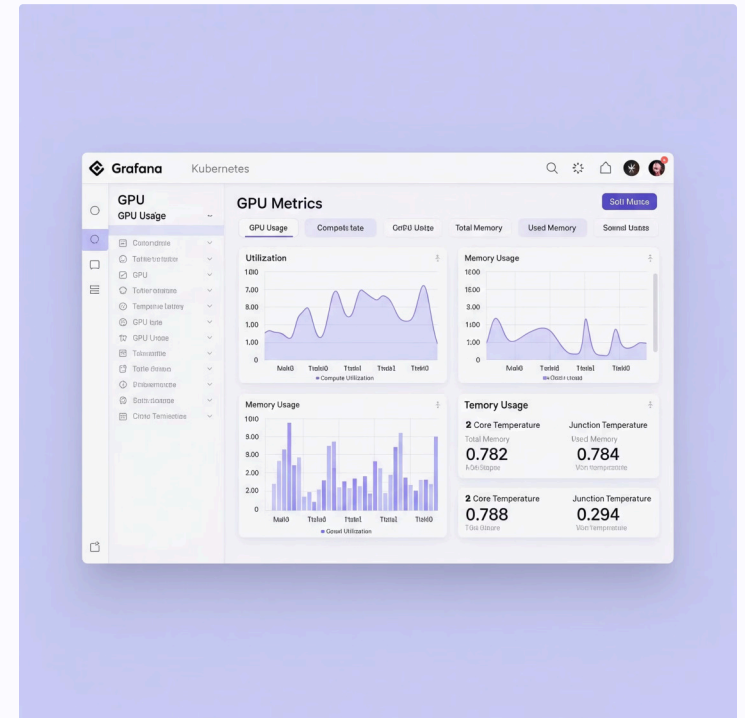
Monitoring GPU Usage in Kubernetes

Basic Monitoring

- `kubectl describe node` shows allocated GPUs
- `kubectl describe pod` shows GPU assignments

Advanced Monitoring

- NVIDIA DCGM Exporter → Prometheus metrics
- Grafana dashboards for GPU utilization
- Custom alerts for idle or overutilized GPUs



Comprehensive monitoring is **critical for cost optimization** and preventing idle GPU waste, which can cost thousands of dollars per month.

Challenges in GPU Scheduling

No Native GPU Preemption

Long-running jobs can block critical workloads, leading to wasted capacity and potential SLA violations

Multi-tenant Fairness

Ensuring equitable GPU access across teams without dedicated hardware silos requires advanced policies

Mixed Workload Inefficiency

Training and inference workloads have different resource patterns but often share the same GPU, causing resource contention

Auto-scaling Complexity

GPU nodes are expensive and slow to provision, making cluster auto-scaling challenging and potentially costly

Advanced schedulers like [KubeFlow](#), [Slurm](#), and [Volcano](#) address some of these challenges but add operational complexity.

Key Takeaways

GPU scheduling is vital for AI infrastructure efficiency

Proper configuration can save thousands of dollars in GPU utilization

NVIDIA device plugin enables K8s GPU support

Required for any GPU workloads in Kubernetes environments

Pods request GPUs via resource limits

Simple syntax for hardware allocation with granular control options

MIG allows fractional GPU sharing

Significantly improves utilization for inference workloads

For production AI clusters, implement comprehensive [monitoring](#), [fairness policies](#), and [resource quotas](#) to maximize GPU infrastructure ROI.

